

FASE EXTRACT — Dokumentasi Pembelajaran ETL Pipeline

File Sumber Kode: `src/extract.py`

Penulis: Dokumentasi Pembelajaran

Terakhir Diperbarui: 2 Juni 2026

Level: Beginner — Intermediate

Daftar Isi

1. Pendahuluan (Apa Itu Extract?)
2. Arsitektur & Alur Kerja Extract Phase
3. Library & Dependensi Yang Digunakan
4. Konfigurasi Logger — Penjelasan Mendalam
5. Fungsi `download_dataset()` — Penjelasan Baris Per Baris
6. Fungsi `find_first_csv()` — Penjelasan Baris Per Baris
7. Fungsi `extract_data_csv()` — Penjelasan Baris Per Baris
8. Fungsi `extract_get_data()` — Orchestrator Utama
9. Fungsi `summary_data()` — Eksplorasi Data Awal
10. Blok `if __name__ == "__main__"` — Penjelasan Mendalam
11. Error Handling Strategy
12. Konsep Python Yang Digunakan
13. Ringkasan & Poin-Poin Penting

1. Pendahuluan (Apa Itu Extract?)

1.1 Definisi Extract dalam Konteks ETL

ETL adalah singkatan dari **Extract, Transform, Load** — tiga tahapan utama dalam memproses data dari sumber mentah hingga menjadi data yang siap digunakan.

Extract (Ekstraksi) adalah **fase pertama** dari pipeline ETL. Secara sederhana, Extract adalah proses **mengambil data mentah (raw data)** dari satu atau lebih sumber data, lalu menyimpannya dalam format yang bisa diproses lebih lanjut

oleh program kita.

Bayangkan analogi ini:

- 🏭 **ETL** = Pabrik pengolahan makanan
- 🍷 **Extract** = Petani mengambil bahan mentah dari ladang
- 🍳 **Transform** = Koki mengolah bahan mentah menjadi masakan
- 🍽️ **Load** = Pelayan menyajikan masakan ke pelanggan (database/warehouse)

Dalam source code kita, Extract berarti:

1. Mencari file CSV di folder `data/`
2. Jika tidak ada, mengunduh dataset dari Kaggle
3. Membaca file CSV tersebut menjadi `pandas DataFrame`
4. Melakukan ringkasan awal terhadap data yang berhasil diekstrak

1.2 Mengapa Extract Adalah Fase Pertama dan Paling Krusial?

Extract adalah fase paling krusial karena beberapa alasan:

| No | Alasan | Penjelasan | |----|-----|-----| | 1 | **Garbage In, Garbage Out** | Jika data yang diekstrak salah atau korup, seluruh pipeline akan menghasilkan output yang salah, tidak peduli seberapa bagus Transform dan Load-nya. | | 2 | **Fondasi Data** | Extract menentukan kualitas awal data. Kesalahan encoding, file kosong, atau format yang salah harus ditangkap di fase ini. | | 3 | **Sumber Kebenaran** | Data yang diekstrak adalah "sumber kebenaran" (source of truth). Kita harus memastikan data ini utuh dan lengkap. | | 4 | **Deteksi Masalah Awal** | Lebih baik gagal di fase Extract (dan tahu penyebabnya) daripada gagal di fase Transform atau Load tanpa tahu akar masalahnya. | | 5 | **Reproducibility** | Extract yang baik memastikan kita bisa mengulang proses yang sama dan mendapat hasil yang konsisten. |

1.3 Sumber Data yang Umum Digunakan

Dalam dunia nyata, data bisa berasal dari berbagai sumber:

Sumber Data	Format	Contoh	Digunakan di Kode Ini?
File CSV	Comma-Separated Values	<code>data.csv</code>	✓ Ya
API (REST/GraphQL)	JSON, XML	Twitter API, Weather API	✗ Tidak
Database (SQL)	Tabel relasional	PostgreSQL, MySQL	✗ Tidak
Web Scraping	HTML	Website berita	✗ Tidak
Cloud Storage	Berbagai format	AWS S3, Google Cloud Storage	✗ Tidak
Platform Dataset	Berbagai format	Kaggle (via <code>kagglehub</code>)	✓ Ya (sebagai fallback)
Spreadsheet	XLSX, XLS	Microsoft Excel	✗ Tidak
Data Streaming	Event stream	Apache Kafka, RabbitMQ	✗ Tidak

Dalam source code ini, kita menggunakan **dua sumber data**:

1. **File CSV lokal** — sumber utama (dari folder `data/`)
2. **Kaggle** — sumber cadangan (jika file CSV lokal tidak tersedia)

1.4 Prinsip-Prinsip Extract yang Baik

Berikut adalah prinsip-prinsip yang diterapkan dalam source code kita:

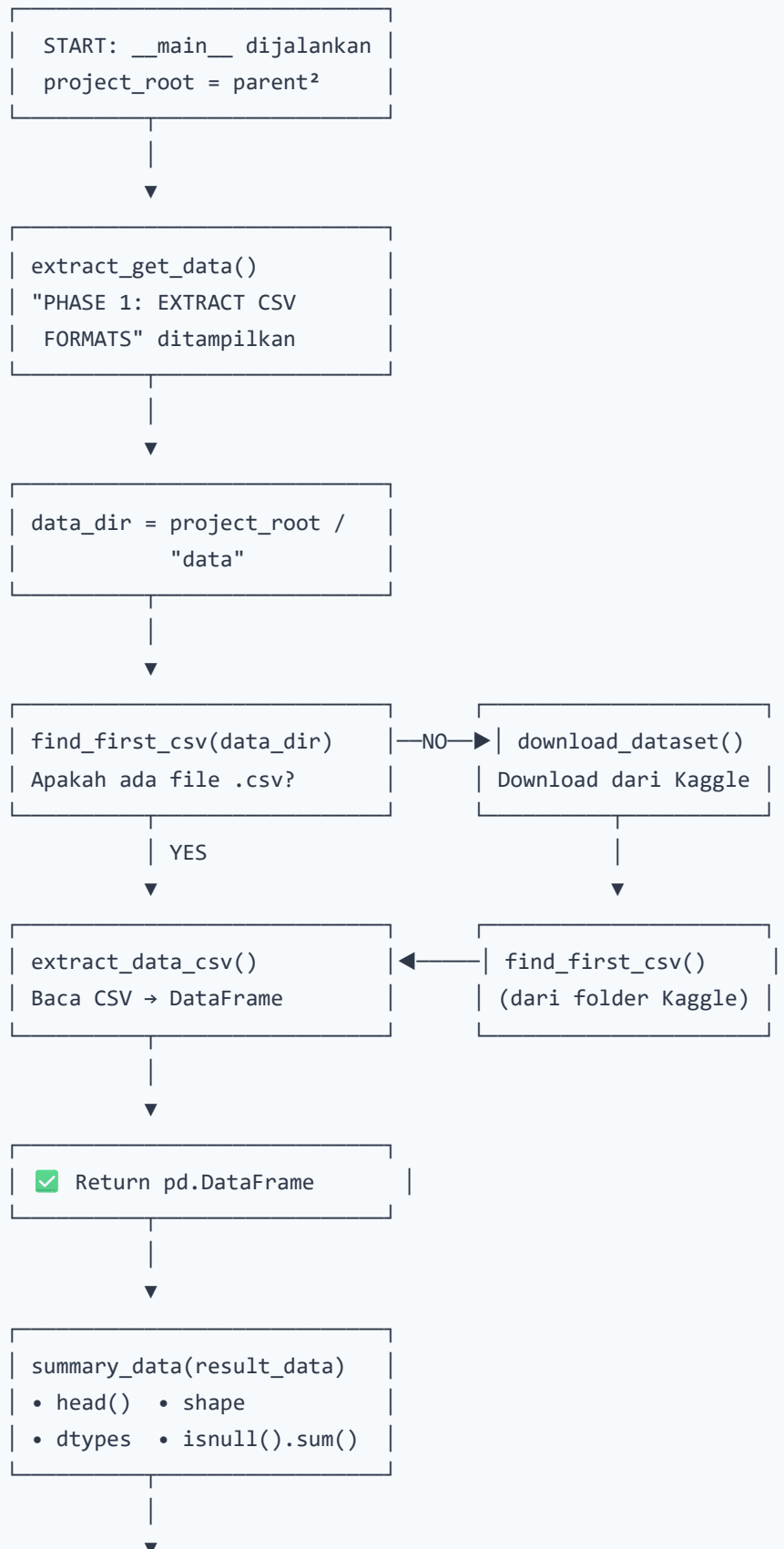
1. 🔍 **Validasi Input** — Selalu periksa apakah file ada dan formatnya benar sebelum membaca
 2. 🛡️ **Error Handling yang Komprehensif** — Tangani semua kemungkinan error (file kosong, encoding salah, permission denied)
 3. 📝 **Logging yang Detail** — Catat setiap langkah untuk memudahkan debugging
 4. 🔄 **Fallback Mechanism** — Punya mekanisme cadangan jika sumber utama gagal (download dari Kaggle)
 5. 📊 **Data Summary** — Lakukan ringkasan awal untuk memverifikasi data yang diekstrak
 6. 🧩 **Modularitas** — Pisahkan setiap tanggung jawab ke fungsi yang berbeda
-

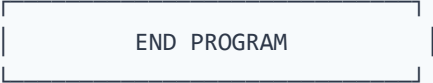
2. 🏗️ Arsitektur & Alur Kerja Extract Phase

2.1 Diagram Alur Keseluruhan

Berikut adalah diagram alur lengkap dari Extract Phase. Setiap langkah menunjukkan apa yang terjadi dari awal program dijalankan hingga data siap digunakan:

EXTRACT PHASE – ALUR KERJA



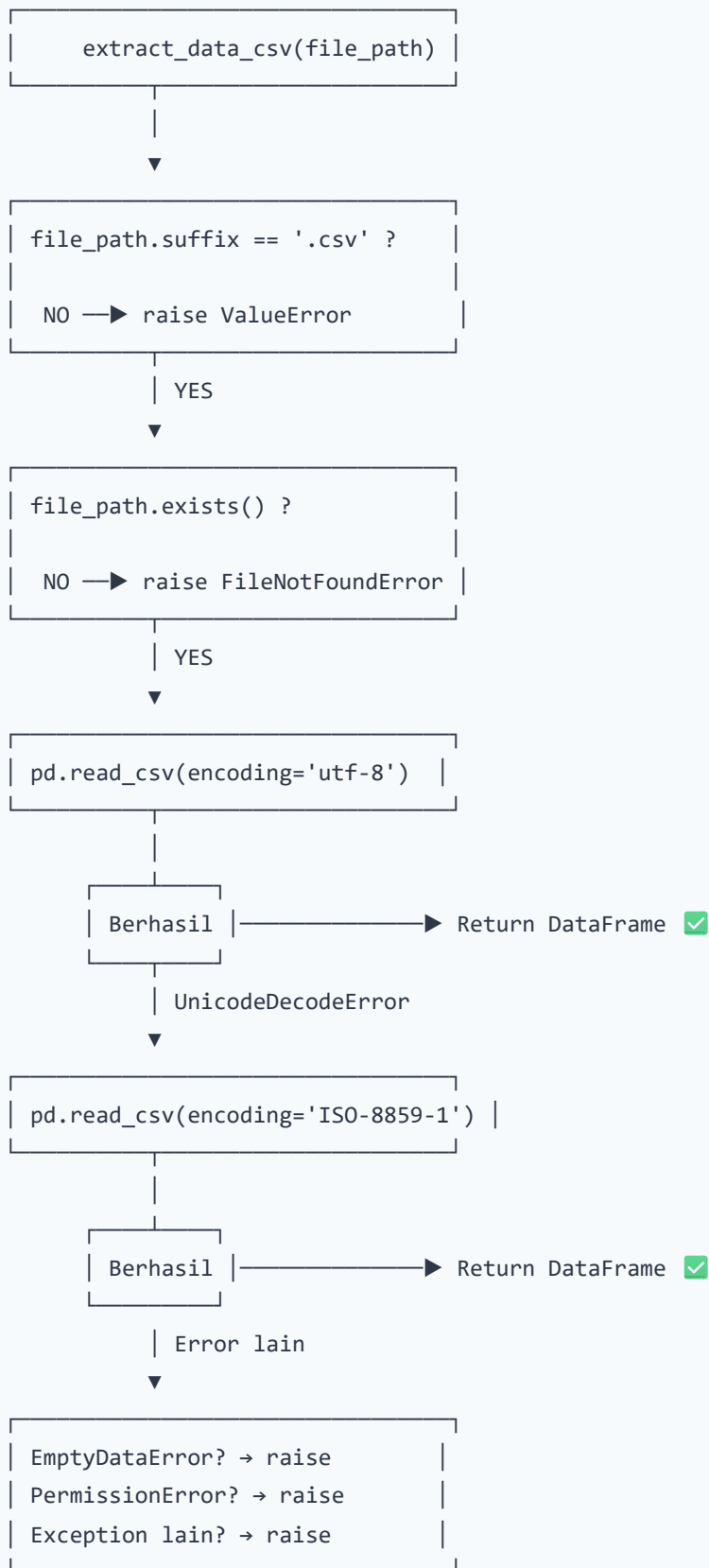


2.2 Penjelasan Setiap Langkah dalam Alur

| Langkah | Fungsi yang Dipanggil | Apa yang Terjadi | Jika Gagal | |-----|-----|-----|-----|
----| | 1 | `__main__` | Menentukan `project_root` dan memulai proses | Program tidak berjalan | | 2 |
`extract_get_data()` | Menampilkan header dan menyiapkan path `data/` | — | | 3 | `find_first_csv()` | Mencari file
`.csv` pertama di folder `data/` | Melempar `ValueError` | | 4 | `download_dataset()` | Download dataset dari Kaggle
(jika CSV tidak ada) | Melempar `FileExistsError`, lalu retry | | 5 | `extract_data_csv()` | Membaca CSV menjadi
DataFrame dengan validasi | Melempar berbagai exception | | 6 | `summary_data()` | Menampilkan ringkasan data (baris,
kolom, missing values) | Return `None` jika data kosong |

2.3 Diagram Alur Extract Data CSV (Detail)

Berikut adalah detail alur internal dari fungsi `extract_data_csv()` :



3. Library & Dependensi Yang Digunakan

Bagian paling atas dari source code kita adalah **import statements** — baris-baris yang memanggil library eksternal:

```
import pandas as pd
import logging
import kagglehub
from pathlib import Path
```

Mari kita bahas satu per satu secara mendalam.

3.1 pandas (`import pandas as pd`)

Apa itu pandas?

`pandas` adalah library Python paling populer untuk **manipulasi dan analisis data**. Library ini menyediakan struktur data yang powerful bernama `DataFrame` — bayangkan seperti spreadsheet Excel yang bisa dimanipulasi dengan kode Python.

Mengapa `as pd` ?

`as pd` adalah **alias** (nama singkat). Ini adalah konvensi standar di komunitas Python. Daripada menulis `pandas.read_csv()`, kita cukup menulis `pd.read_csv()`. Lebih pendek dan semua programmer Python akan langsung mengerti.

Fungsi pandas yang digunakan di source code ini:

Fungsi Kegunaan Baris di Kode	<code>pd.read_csv()</code> Membaca file CSV menjadi DataFrame Baris 91, 94	<code>pd.errors.EmptyDataError</code> Exception jika file CSV kosong Baris 100	<code>df.head()</code> Menampilkan 5 baris pertama data Baris 176	<code>df.shape</code> Mengembalikan tuple (jumlah_baris, jumlah_kolom) Baris 177	<code>df.dtypes</code> Menampilkan tipe data setiap kolom Baris 179	<code>df.isnull().sum()</code> Menghitung jumlah missing values per kolom Baris 182
-----------------------------------	--	--	---	--	---	---

Contoh penggunaan:

```
# Membaca CSV menjadi DataFrame
df = pd.read_csv("data.csv", encoding='utf-8')

# Melihat 5 baris pertama
print(df.head())
#      InvoiceNo StockCode  Quantity  InvoiceDate  UnitPrice  CustomerID
# 0      536365      85123A         6  12/1/2010 8:26         2.55      17850.0
# 1      536365      71053         6  12/1/2010 8:26         3.39      17850.0
# ...

# Melihat dimensi data
print(df.shape) # (541909, 8) → 541.909 baris, 8 kolom
```

3.2 📄 logging (`import logging`)

Apa itu logging?

`logging` adalah modul bawaan Python (standard library) untuk **mencatat pesan/peristiwa** selama program berjalan. Berbeda dengan `print()` yang hanya menampilkan teks, `logging` memberikan informasi tambahan seperti waktu, level keparahan, file sumber, dan nomor baris.

Mengapa logging penting dalam ETL Pipeline?

| Aspek | `print()` | `logging` | |-----|-----|-----| | Level keparahan | ❌ Tidak ada | ✅ DEBUG, INFO, WARNING, ERROR, CRITICAL | | Timestamp | ❌ Tidak ada | ✅ Otomatis ditambahkan | | File & Line info | ❌ Tidak ada | ✅ Otomatis ditambahkan | | Output ke file | ❌ Sulit | ✅ Mudah dikonfigurasi | | Bisa dimatikan | ❌ Harus hapus manual | ✅ Cukup ubah level | | Produksi ready | ❌ Tidak cocok | ✅ Standar industri |

Level-level Logging:

Logging memiliki 5 level keparahan, dari yang paling ringan hingga paling berat:

Level	Angka	Kapan Digunakan
DEBUG	10	Informasi detail untuk debugging. Tidak untuk produksi
INFO	20	Konfirmasi bahwa sesuatu berjalan sesuai harapan
WARNING	30	Sesuatu yang tidak terduga terjadi, tapi program masih bisa berjalan
ERROR	40	Masalah serius, program tidak bisa menjalankan suatu fungsi
CRITICAL	50	Masalah fatal, program mungkin harus berhenti

Dalam source code kita, yang digunakan:

- `logger.info()` — ✅ Proses berhasil (baris 26, 35, 55, 88, 95, 150, 177)
- `logger.warning()` — ⚠️ Ada masalah kecil tapi ditangani (baris 93, 143)
- `logger.error()` — ❌ Ada masalah besar (baris 53, 78, 83, 101, 106, 111)

3.3 📦 kagglehub (`import kagglehub`)

Apa itu kagglehub?

`kagglehub` adalah library resmi dari **Kaggle** (platform data science terbesar di dunia) untuk mengunduh dataset langsung dari Python. Ini lebih modern dan lebih mudah dibanding pendahulunya (`kaggle` CLI tool).

Fungsi utama yang digunakan:

```
kagglehub.dataset_download(dataset, output_dir=str(output_dir), force_download=force_download)
```

Parameter	Tipe	Penjelasan
<code>dataset</code>	<code>str</code>	Nama dataset di Kaggle, formatnya "username/dataset-name"
<code>output_dir</code>	<code>str</code>	Direktori tujuan download (opsional)
<code>force_download</code>	<code>bool</code>	Jika <code>True</code> , download ulang meskipun sudah ada

Contoh:

```
# Download dataset e-commerce dari user "carrie1"
path = kagglehub.dataset_download("carrie1/ecommerce-data", output_dir="./data")
# path sekarang menunjuk ke folder tempat dataset disimpan
```

3.4 📁 pathlib — Path (`from pathlib import Path`)

Apa itu pathlib?

`pathlib` adalah modul bawaan Python (sejak Python 3.4) yang menyediakan cara **object-oriented** untuk bekerja dengan path file dan direktori. `Path` adalah class utama dalam modul ini.

Mengapa Path lebih baik dari `os.path` ?

Fitur	<code>os.path</code> (cara lama)	<code>pathlib.Path</code> (cara modern)
Menggabungkan path	<code>os.path.join("data", "file.csv")</code>	<code>Path("data") / "file.csv"</code>
Cek file ada	<code>os.path.exists("file.csv")</code>	<code>Path("file.csv").exists()</code>
Ambil ekstensi	<code>os.path.splitext("file.csv")[1]</code>	<code>Path("file.csv").suffix</code>
Ambil nama file	<code>os.path.basename("/a/b/file.csv")</code>	<code>Path("/a/b/file.csv").name</code>
Glob pattern	<code>import glob; glob.glob("*.csv")</code>	<code>Path(".").glob("*.csv")</code>
Readability	🙄 Prosedural, panjang	😊 OOP, pendek, intuitif

Operasi Path yang digunakan di source code:

```
# Menggabungkan path dengan operator /
data_dir = project_root / "data"          # Baris 133

# Mendapatkan ekstensi file
file_path.suffix                          # Baris 77 → '.csv'

# Cek apakah file ada
file_path.exists()                        # Baris 82

# Mendapatkan nama file saja
file_path.name                            # Baris 88 → 'data.csv'

# Glob: mencari file dengan pola tertentu
dataset_path.glob("*.csv")                # Baris 51

# Mendapatkan parent directory
Path(__file__).resolve().parent.parent    # Baris 189
```

4. 🛠️ Konfigurasi Logger — Penjelasan Mendalam

4.1 Kode Konfigurasi Logger

Berikut adalah kode konfigurasi logger yang ada di source code:

```
# Get the logger, for checking process on terminal
logger = logging.getLogger(__name__)

# basic config for logger
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(filename)s - %(lineno)d - %(levelname)s - %(message)s"
)
```

4.2 `logging.getLogger(__name__)` — Membuat Logger

```
logger = logging.getLogger(__name__)
```

Apa yang terjadi?

- `logging.getLogger()` membuat atau mengambil sebuah **logger object** — yaitu objek yang bertugas mencatat pesan
- `__name__` adalah variabel bawaan Python yang berisi **nama modul** saat ini
 - Jika file dijalankan langsung: `__name__` = `"__main__"`
 - Jika file diimport dari file lain: `__name__` = `"extract"` (nama modul)
- Menggunakan `__name__` membantu kita **mengidentifikasi dari mana log berasal** saat kita punya banyak file

4.3 `logging.basicConfig()` — Mengatur Format dan Level

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(filename)s - %(lineno)d - %(levelname)s - %(message)s"
)
```

Parameter `level=logging.INFO` :

Ini berarti logger hanya akan menampilkan pesan dengan level **INFO ke atas** (INFO, WARNING, ERROR, CRITICAL). Pesan DEBUG akan **diabaikan**.

DEBUG	→	✗	Tidak ditampilkan (di bawah INFO)
INFO	→	✓	Ditampilkan
WARNING	→	✓	Ditampilkan
ERROR	→	✓	Ditampilkan
CRITICAL	→	✓	Ditampilkan

4.4 Penjelasan Setiap Bagian Format String

Format string `"%(asctime)s - %(name)s - %(filename)s - %(lineno)d - %(levelname)s - %(message)s"` terdiri dari beberapa **placeholder** yang akan diganti dengan informasi aktual saat log dicetak:

Placeholder	Tipe	Penjelasan	Contoh Output
<code>%(asctime)s</code>	s (string)	Timestamp — waktu kapan log dibuat	<code>2026-06-02 00:30:15,123</code>
<code>%(name)s</code>	s (string)	Nama logger — dari <code>getLogger(__name__)</code>	<code>__main__</code> atau <code>extract</code>
<code>%(filename)s</code>	s (string)	Nama file tempat log dipanggil	<code>extract.py</code>
<code>%(lineno)d</code>	d (integer)	Nomor baris tempat log dipanggil	<code>26</code>
<code>%(levelname)s</code>	s (string)	Level log — INFO, WARNING, ERROR, dll	<code>INFO</code>
<code>%(message)s</code>	s (string)	Pesan yang kita tulis	

[Download Dataset From Kaggle](#)

💡 **Catatan:** Huruf `s` setelah `%` artinya format **string**, sedangkan `d` artinya format **decimal (integer)**. Inilah mengapa `%(lineno)d` menggunakan `d` — karena nomor baris adalah angka.

4.5 Contoh Output Log

Ketika program berjalan, berikut contoh output log yang akan muncul di terminal:

```
2026-06-02 00:30:15,123 - __main__ - extract.py - 26 - INFO - 📄 Download Dataset From Kaggle
2026-06-02 00:30:18,456 - __main__ - extract.py - 35 - INFO - ✓ Download Dataset Kaggle is successful
2026-06-02 00:30:18,457 - __main__ - extract.py - 55 - INFO - ✓ Found CSV file: data/data.csv
2026-06-02 00:30:18,458 - __main__ - extract.py - 88 - INFO - 📄 Reading CSV Files : data.csv
2026-06-02 00:30:19,789 - __main__ - extract.py - 95 - INFO - ✓ Read Data from data.csv is successfully
2026-06-02 00:30:19,790 - __main__ - extract.py - 150 - INFO - ✓ EXTRACT AND READ CSV FILES IS SUCCESSFU
2026-06-02 00:30:19,800 - __main__ - extract.py - 177 - INFO - 📊 Dataset Loaded - Rows: 541909, Columns
```

Perhatikan bagaimana setiap baris log mengandung:

1. **Kapan** terjadi (timestamp)
2. **Dari modul mana** (`__main__`)
3. **Di file apa** (`extract.py`)
4. **Di baris berapa** (26, 35, 55, dst.)
5. **Level apa** (INFO)
6. **Pesan apa** (deskripsi proses)

4.6 Mengapa Logging Penting dalam ETL Pipeline?

Dalam ETL pipeline, logging sangat penting karena:

1. **Audit Trail** — Kita bisa melacak kapan data diproses, berapa lama, dan apakah berhasil
2. **Debugging** — Jika terjadi error, log menunjukkan **persis di mana** masalah terjadi
3. **Monitoring** — Dalam produksi, log bisa dikirim ke sistem monitoring (seperti ELK Stack, Grafana)
4. **Compliance** — Beberapa regulasi mengharuskan pencatatan setiap proses data
5. **Performance Tracking** — Dengan timestamp, kita bisa mengukur berapa lama setiap tahap berjalan

5. Fungsi `download_dataset()` — Penjelasan Baris Per Baris

5.1 Kode Lengkap

```
def download_dataset(dataset:str, output_dir: Path, force_download: bool = False) -> pd.DataFrame:
    """Download Dataset From Kaggle to target directory

    Args:
        dataset (str): The name of the dataset to download.
        output_dir (Path): The path to the directory where the dataset will be downloaded.
        force_download (bool): Whether to force the download of the dataset. When the dataset exists

    Returns:
        Path: The path to the downloaded dataset.
    """
    logger.info(f" Download Dataset From Kaggle")

    # Check path directory "data". If there is any, the data will be saved there.
    if output_dir:
        path_dataset = Path(kagglehub.dataset_download(dataset, output_dir=str(output_dir), force_download=force_download))

    # Else, it will be saved in the default directory.
    else:
        path_dataset = Path(kagglehub.dataset_download(dataset, force_download=force_download))
    logger.info(f"✓ Download Dataset Kaggle is successful")
    return path_dataset
```

5.2 Signature Fungsi (Baris Pertama)

```
def download_dataset(dataset:str, output_dir: Path, force_download: bool = False) -> pd.DataFrame:
```

Mari kita bedah setiap bagian:

5.2.1 Type Hints

Type hints adalah anotasi tipe yang memberitahu programmer (dan IDE) tipe data apa yang diharapkan. Type hints **tidak memaksa** Python untuk menolak tipe yang salah — ini hanya "petunjuk" untuk manusia dan tools.

Parameter	Type Hint	Penjelasan	-----	-----	-----		<code>dataset</code>	<code>str</code>	String — nama dataset di Kaggle, contoh: <code>"carrie1/ecommerce-data"</code>	<code>output_dir</code>	<code>Path</code>	Objek Path dari <code>pathlib</code> — path tujuan
-----------	-----------	------------	-------	-------	-------	--	----------------------	------------------	--	-------------------------	-------------------	--

download || `force_download` | `bool` | Boolean — `True` atau `False` , apakah paksa download ulang || Return value |
-> `pd.DataFrame` | Menunjukkan fungsi mengembalikan DataFrame (⚠️ sebenarnya mengembalikan `Path`) |

⚠️ **Catatan Penting:** Ada ketidaksesuaian (mismatch) di type hint return value. Fungsi ini sebenarnya mengembalikan `Path` , bukan `pd.DataFrame` . Docstring-nya sudah benar menuliskan `Returns: Path` . Ini adalah hal yang bisa diperbaiki di masa depan.

5.2.2 Default Parameter

```
force_download: bool = False
```

Apa itu default parameter?

Default parameter adalah parameter yang **sudah memiliki nilai bawaan**. Jika pemanggil fungsi tidak memberikan nilai untuk parameter ini, maka nilai default-nya yang akan digunakan.

```
# Tanpa memberikan force_download → otomatis False
download_dataset("carrie1/ecommerce-data", output_dir=data_dir)
# Sama dengan:
download_dataset("carrie1/ecommerce-data", output_dir=data_dir, force_download=False)

# Dengan memberikan force_download = True
download_dataset("carrie1/ecommerce-data", output_dir=data_dir, force_download=True)
```

Mengapa `False` sebagai default?

Karena dalam kebanyakan kasus, kita **tidak ingin** mengunduh ulang dataset yang sudah ada. Download ulang membuang waktu dan bandwidth. Hanya dalam kasus khusus (misalnya data korup) kita perlu memaksa download ulang.

5.3 Docstring

```
"""Download Dataset From Kaggle to target directory
```

Args:

dataset (str): The name of the dataset to download.

output_dir (Path): The path to the directory where the dataset will be downloaded.

force_download (bool): Whether to force the download of the dataset. When the dataset exists

Returns:

Path: The path to the downloaded dataset.

```
"""
```

Apa itu docstring?

Docstring adalah **string dokumentasi** yang ditulis di awal fungsi, class, atau modul. Diapit oleh **triple quotes** (`"""`). Docstring menjelaskan:

- Apa yang dilakukan fungsi ini
- Parameter apa yang diterima (`Args`)
- Apa yang dikembalikan (`Returns`)
- Exception apa yang bisa terjadi (`Raises`) — tidak ada di sini

Cara membaca docstring:

```
# Di Python interpreter atau Jupyter Notebook:
help(download_dataset)
# Atau:
print(download_dataset.__doc__)
```

Format yang digunakan di sini adalah **Google-style docstring** — salah satu format paling populer dan mudah dibaca.

5.4 Logika If-Else untuk `output_dir`

```
if output_dir:
    path_dataset = Path(kagglehub.dataset_download(dataset, output_dir=str(output_dir), force_download=force_download))
else:
    path_dataset = Path(kagglehub.dataset_download(dataset, force_download=force_download))
```

Apa yang terjadi?

- `if output_dir:` — Mengecek apakah `output_dir` memiliki nilai (bukan `None`, bukan string kosong, bukan `False`). Dalam Python, objek Path yang valid akan dianggap sebagai **truthy**.
- Jika `output_dir` **ada** (truthy): Download ke direktori yang ditentukan
- Jika `output_dir` **tidak ada** (falsy): Download ke direktori default kagglehub (biasanya `~/.cache/kagglehub/`)

Mengapa `str(output_dir)` ?

Karena `output_dir` bertipe `Path`, tapi `kagglehub.dataset_download()` mengharapkan parameter `output_dir` dalam bentuk **string**. Jadi kita perlu mengkonversinya dengan `str()`.

5.5 Return Value

```
return path_dataset
```

Fungsi mengembalikan objek `Path` yang menunjuk ke **direktori tempat dataset disimpan**. Path ini akan digunakan oleh fungsi lain (seperti `find_first_csv()`) untuk mencari file CSV di dalamnya.

6. Fungsi `find_first_csv()` — Penjelasan Baris Per Baris

6.1 Kode Lengkap

```
def find_first_csv(dataset_path: Path) -> pd.DataFrame:
    csv_files = list(dataset_path.glob("*.csv"))
    if not csv_files:
        logger.error("⚠ No CSV files found in dataset path: %s", dataset_path)
        raise ValueError("No CSV files found in dataset path")
    logger.info("✓ Found CSV file: %s", csv_files[0])
    return csv_files[0]
```

6.2 `dataset_path.glob("*.csv")` — Glob Pattern

```
csv_files = list(dataset_path.glob("*.csv"))
```

Apa itu glob pattern?

Glob pattern adalah **pola pencarian file** menggunakan karakter wildcard. Berasal dari sistem Unix, glob digunakan untuk mencocokkan nama file berdasarkan pola.

Penjelasan `*.csv` :

| Bagian | Arti | |-----|-----| | `*` | Wildcard — cocokkan **karakter apa saja** dalam jumlah berapa saja (termasuk nol) | | `.csv` | Literal — harus diakhiri dengan `.csv` |

Contoh file yang cocok dengan `*.csv` :

- ✓ data.csv
- ✓ ecommerce-data.csv
- ✓ 123.csv
- ✓ a.csv
- ✗ data.txt (bukan .csv)
- ✗ data.csv.bak (.bak bukan .csv)
- ✗ subfolder/data.csv (glob tanpa ** hanya cari di level saat ini)

Variasi glob pattern lainnya (untuk pengetahuan):

| Pattern | Arti | Contoh yang Cocok | |-----|-----|-----| | `*.csv` | Semua file .csv di level ini | data.csv , test.csv | | `**/*.csv` | Semua file .csv di semua subdirektori (rekursif) | sub/data.csv , a/b/c.csv | | `data_*` | Semua file yang diawali data_ | data_2024.csv , data_test.txt | | `?.csv` | File .csv dengan nama 1 karakter | a.csv , 1.csv |

6.3 Mengapa Menggunakan `list()` untuk Membungkus Glob?

```
csv_files = list(dataset_path.glob("*.csv"))
```

`dataset_path.glob("*.csv")` mengembalikan sebuah **generator** — yaitu objek yang menghasilkan nilai satu per satu secara **lazy** (malas). Generator **tidak menyimpan semua hasil di memori** sekaligus.

Mengapa perlu `list()` ?

| Tanpa `list()` (Generator) | Dengan `list()` (List) | |-----|-----| | Hanya bisa diiterasi **sekali** | Bisa diiterasi **berkali-kali** | | Tidak bisa diakses dengan index `[0]` | Bisa diakses: `csv_files[0]` ✓ | | Tidak bisa dicek `len()` langsung | Bisa: `len(csv_files)` ✓ | | Tidak bisa dicek `if not ...` dengan benar | Bisa: `if not csv_files:` ✓ |

Karena kita perlu:

1. Mengecek apakah list kosong (`if not csv_files`)
2. Mengambil elemen pertama (`csv_files[0]`)

...maka kita **perlu** mengkonversi generator menjadi list.

6.4 Pengecekan `if not csv_files`

```
if not csv_files:
    logger.error("⚠️ No CSV files found in dataset path: %s", dataset_path)
    raise ValueError("No CSV files found in dataset path")
```

Apa artinya `if not csv_files` ?

Dalam Python, sebuah **list kosong** (`[]`) dianggap **falsy** — artinya dievaluasi sebagai `False` dalam konteks boolean.

```
csv_files = []          # List kosong
if not csv_files:       # not False → True
    print("Tidak ada file CSV!") # Ini akan dijalankan

csv_files = ["data.csv"] # List berisi
if not csv_files:       # not True → False
    print("Tidak ada!")   # Ini TIDAK dijalankan
```

Truthiness di Python — Quick Reference:

| Nilai | Truthy/Falsy | |-----|-----| | `[]` (list kosong) | Falsy | | `["data.csv"]` (list berisi) | Truthy | | `""` (string kosong) | Falsy | | `"hello"` (string berisi) | Truthy | | `0` | Falsy | | `42` | Truthy | | `None` | Falsy | | `Path("data")` (objek Path) | Truthy |

6.5 `raise ValueError` — Apa Itu Exception?

```
raise ValueError("No CSV files found in dataset path")
```

Apa itu Exception?

Exception adalah **mekanisme penanganan error** di Python. Ketika sesuatu yang tidak diharapkan terjadi, program "melempar" (raise) sebuah exception. Exception ini bisa "ditangkap" (catch) oleh kode yang memanggilnya menggunakan `try-except`.

Apa itu `raise` ?

`raise` adalah keyword Python yang **secara sengaja melempar exception**. Ini berarti: "ada masalah, dan saya ingin memberitahu pemanggil fungsi ini."

Apa itu `ValueError` ?

`ValueError` adalah tipe exception bawaan Python yang digunakan ketika sebuah fungsi menerima **argumen dengan nilai yang tidak valid**. Dalam konteks ini: folder dataset ada, tapi **tidak berisi file CSV** — nilainya tidak sesuai harapan.

Apa yang terjadi setelah `raise` ?

```
# Eksekusi program BERHENTI di baris ini
raise ValueError("No CSV files found in dataset path")

# Baris-baris di bawah TIDAK AKAN dijalankan
logger.info("✓ Found CSV file: %s", csv_files[0]) # ← TIDAK dijalankan
return csv_files[0]                               # ← TIDAK dijalankan
```

Program akan "naik" ke atas (propagate) mencari blok `try-except` terdekat. Jika tidak ada yang menangkap, program akan crash dengan pesan error.

7. Fungsi `extract_data_csv()` — Penjelasan Baris Per Baris

7.1 Kode Lengkap

```
def extract_data_csv(file_path: Path) -> pd.DataFrame:
    if file_path.suffix != '.csv':
        logger.error("❌ File path does not end with CSV Formats")
        raise ValueError("Invalid File Format")

    if not file_path.exists():
        logger.error("❌ File path does not exist")
        raise FileNotFoundError("File Path Does Not Exist")

    try:
        logger.info(f"📖 Reading CSV Files : {file_path.name}")
        try:
            df = pd.read_csv(file_path, encoding='utf-8')
        except UnicodeDecodeError:
            logger.warning(f"⚠️ UTF-8 decoding failed for {file_path.name}, retrying with ISO-8859-1")
            df = pd.read_csv(file_path, encoding='ISO-8859-1')
        logger.info(f"✅ Read Data from {file_path.name} is successfully")
        return df

    except pd.errors.EmptyDataError:
        logger.error(f"❌ File is empty: {file_path.name}")
        raise

    except PermissionError:
        logger.error(f"❌ Permission Denied for this file: {file_path.name}")
        raise

    except Exception as e:
        logger.error(f"❌ Another Error to read CSV file: {str(e)}")
        raise
```

7.2 Validasi File — `.suffix` dan `.exists()`

`.suffix` — Mengecek Ekstensi File

```
if file_path.suffix != '.csv':
    logger.error("❌ File path does not end with CSV Formats")
    raise ValueError("Invalid File Format")
```

Apa itu `.suffix` ?

`.suffix` adalah properti dari objek `Path` yang mengembalikan **ekstensi file** (termasuk titiknya).

```
Path("data.csv").suffix      # → '.csv'
Path("report.xlsx").suffix   # → '.xlsx'
Path("archive.tar.gz").suffix # → '.gz' (hanya ekstensi terakhir!)
Path("README").suffix        # → '' (string kosong, tidak ada ekstensi)
```

Mengapa validasi ini penting?

Karena `pd.read_csv()` mengharapkan file berformat CSV. Jika kita mencoba membaca file Excel (.xlsx) atau JSON (.json) dengan `pd.read_csv()`, hasilnya akan error atau data yang korup. Lebih baik **gagal lebih awal** (fail fast) dengan pesan error yang jelas.

`.exists()` — Mengecek Keberadaan File

```
if not file_path.exists():
    logger.error("❌ File path does not exist")
    raise FileNotFoundError("File Path Does Not Exist")
```

Apa itu `.exists()` ?

`.exists()` adalah method dari objek `Path` yang mengembalikan `True` jika file atau direktori **benar-benar ada** di sistem file, dan `False` jika tidak.

```
Path("data/data.csv").exists()      # True (jika file ada)
Path("data/nonexistent.csv").exists() # False (file tidak ada)
```

Mengapa validasi ini penting?

Tanpa pengecekan ini, `pd.read_csv()` akan melempar `FileNotFoundError` juga — tapi pesan error-nya mungkin tidak sejelas yang kita buat sendiri. Dengan validasi manual, kita bisa:

1. Mencatat error ke log dengan pesan yang deskriptif
2. Memberikan pesan error yang spesifik

7.3 Nested Try-Except — Try di Dalam Try

```
try:
    logger.info(f"📄 Reading CSV Files : {file_path.name}")
    try:
        df = pd.read_csv(file_path, encoding='utf-8')
    except UnicodeDecodeError:
        logger.warning(f"⚠️ UTF-8 decoding failed for {file_path.name}, retrying with ISO-8859-1")
        df = pd.read_csv(file_path, encoding='ISO-8859-1')
    logger.info(f"✓ Read Data from {file_path.name} is successfully")
    return df
except pd.errors.EmptyDataError:
    ...
except PermissionError:
    ...
except Exception as e:
    ...
```

Mengapa ada try di dalam try?

Karena kita ingin menangani **jenis error yang berbeda pada level yang berbeda**:

```
try LUAR:
    | Menangani error "besar":
    | - EmptyDataError (file kosong)
    | - PermissionError (tidak punya akses)
    | - Exception umum lainnya
    |
    └─ try DALAM:
        Menangani error "kecil" yang BISA dipulihkan:
        - UnicodeDecodeError (encoding salah → coba encoding lain)
```

Alur logika:

1. **Try dalam** mencoba baca CSV dengan UTF-8
2. Jika **UTF-8 gagal** (UnicodeDecodeError): tangkap, log warning, coba lagi dengan ISO-8859-1
3. Jika **ISO-8859-1 juga gagal**: error akan "naik" ke **try luar**
4. **Try luar** menangani error fatal yang tidak bisa dipulihkan

Ini adalah pola yang disebut **graceful degradation** — mencoba cara terbaik dulu, lalu turun ke alternatif jika gagal.

7.4 Encoding: Perbedaan UTF-8 vs ISO-8859-1

Apa itu Encoding?

Encoding adalah cara komputer **mengkonversi karakter (huruf, angka, simbol) menjadi angka (byte)**. Setiap encoding memiliki "kamus" yang berbeda.

Mengapa ada masalah encoding?

Karena ada banyak sistem encoding yang berbeda di dunia, dan file CSV bisa dibuat dengan encoding apa saja. Jika kita mencoba membaca file dengan encoding yang **salah**, karakter tertentu (terutama karakter non-ASCII seperti é, ñ, ü) bisa menjadi rusak atau menyebabkan error.

Aspek UTF-8 ISO-8859-1 (Latin-1)	----- ----- -----	Cakupan Semua karakter di dunia (Unicode) Hanya karakter Eropa Barat
Ukuran 1-4 byte per karakter	Selalu 1 byte per karakter	Popularitas 🏆
Standar modern, paling umum	🤖 Masih sering ditemui di data lama	Emoji ✅ Didukung ❌ Tidak didukung
Karakter Asia ✅ Didukung ❌ Tidak didukung	Backward compatible	✅ Dengan ASCII ✅ Dengan ASCII
Error jika salah Bisa <code>UnicodeDecodeError</code>	Jarang error (menerima semua byte 0-255)	

Mengapa fallback ke ISO-8859-1?

ISO-8859-1 **tidak pernah gagal** membaca file karena setiap byte (0-255) adalah karakter valid. Jadi sebagai fallback, ini sangat aman. Hasilnya mungkin tidak sempurna untuk karakter non-Latin, tapi setidaknya data **bisa dibaca**.

```
# Percobaan 1: UTF-8 (terbaik)
try:
    df = pd.read_csv(file_path, encoding='utf-8')
except UnicodeDecodeError:
    # Percobaan 2: ISO-8859-1 (fallback aman)
    df = pd.read_csv(file_path, encoding='ISO-8859-1')
```

7.5 `pd.read_csv()` — Parameter Penting

```
df = pd.read_csv(file_path, encoding='utf-8')
```

`pd.read_csv()` adalah fungsi utama pandas untuk membaca file CSV. Berikut parameter yang digunakan dan parameter penting lainnya:

Parameter Nilai di Kode Penjelasan	----- ----- -----	<code>filepath_or_buffer</code> <code>file_path</code>
Path ke file CSV (wajib, argumen pertama)	<code>encoding</code> <code>'utf-8'</code> atau <code>'ISO-8859-1'</code>	Encoding karakter file

Parameter penting lainnya yang tidak digunakan tapi perlu diketahui:

Parameter	Default	Penjelasan
<code>sep</code>	<code>','</code>	Pemisah kolom (bisa <code>','</code> , <code>'\t'</code> untuk TSV)
<code>header</code>	<code>'infer'</code>	Baris mana yang jadi header (nama kolom)
<code>index_col</code>	<code>None</code>	Kolom mana yang jadi index
<code>usecols</code>	<code>None</code>	Kolom mana saja yang mau dibaca
<code>dtype</code>	<code>None</code>	Tipe data spesifik per kolom
<code>na_values</code>	<code>None</code>	Nilai apa yang dianggap NaN/missing
<code>nrows</code>	<code>None</code>	Berapa baris yang mau dibaca (untuk testing)
<code>skiprows</code>	<code>None</code>	Baris mana yang dilewati
<code>parse_dates</code>	<code>False</code>	Kolom mana yang diparsing sebagai datetime

7.6 Error Handling Lengkap

Fungsi ini menangani **4 jenis error**:

```
# Error 1: File kosong (tidak ada data)
except pd.errors.EmptyDataError:
    logger.error(f"❌ File is empty: {file_path.name}")
    raise

# Error 2: Tidak punya izin akses file
except PermissionError:
    logger.error(f"❌ Permission Denied for this file: {file_path.name}")
    raise

# Error 3: Encoding gagal (ditangani di try dalam)
except UnicodeDecodeError:
    logger.warning(f"⚠️ UTF-8 decoding failed for {file_path.name}, retrying with ISO-8859-1")
    df = pd.read_csv(file_path, encoding='ISO-8859-1')

# Error 4: Error lainnya yang tidak terduga
except Exception as e:
    logger.error(f"❌ Another Error to read CSV file: {str(e)}")
    raise
```

Error	Kapan Terjadi	Bisa Dipulihkan?	Aksi
<code>UnicodeDecodeError</code>	File tidak ber-encoding UTF-8	✅ Ya	Coba ISO-8859-1 <code>pd.errors.EmptyDataError</code> File CSV ada tapi kosong ❌
<code>PermissionError</code>	OS menolak akses file	❌ Tidak	Log error, re-raise <code>Exception</code>
Exception	Error tak terduga lainnya	❌ Tidak	Log error, re-raise

7.7 Mengapa Menggunakan `raise` Tanpa Argumen?

```
except pd.errors.EmptyDataError:
    logger.error(f"❌ File is empty: {file_path.name}")
    raise # ← tanpa argumen!
```


`raise` tanpa argumen akan **melempar kembali (re-raise) exception yang sama** yang baru saja ditangkap. Ini berbeda dari `raise ValueError("pesan baru")` yang membuat exception baru.

Mengapa re-raise?

Karena kita ingin:

1. **Mencatat error ke log** (agar kita tahu apa yang terjadi)
2. **Tetap melempar error ke pemanggil** (agar pemanggil bisa menanganinya juga)

Jika kita hanya menulis `logger.error(...)` tanpa `raise`, program akan **diam-diam melanjutkan** seolah tidak ada error — ini sangat berbahaya dalam pipeline ETL karena bisa menghasilkan data yang tidak lengkap.

```
# ❌ SALAH – Error "ditelan" diam-diam
except pd.errors.EmptyDataError:
    logger.error("File kosong")
    # Program lanjut tanpa data... BERBAHAYA!

# ✅ BENAR – Error dicatat DAN diteruskan
except pd.errors.EmptyDataError:
    logger.error("File kosong")
    raise # Pemanggil akan tahu ada masalah
```

Perbedaan `raise` vs `raise Exception(...)` vs `raise ... from ...` :

```
# 1. Re-raise exception yang sama (PRESERVASI traceback asli)
except SomeError:
    raise

# 2. Raise exception baru (traceback baru)
except SomeError:
    raise ValueError("Pesan baru")

# 3. Chained exception (traceback baru TAPI terhubung ke asli)
except SomeError as e:
    raise ValueError("Pesan baru") from e
```

8. 🎵 Fungsi `extract_get_data()` — Orchestrator Utama

8.1 Kode Lengkap

```
def extract_get_data(project_root: Path) -> pd.DataFrame:
    print("="*60)
    print("PHASE 1: EXTRACT CSV FORMATS")
    print("="*60)

    data_dir = project_root / "data"
    csv_path = find_first_csv(data_dir)

    if not csv_path:
        try:
            dataset_path = download_dataset("carrie1/ecommerce-data", output_dir=data_dir)
        except FileExistsError:
            logger.warning("⚠️ Folder data is not empty. Retrying download with force_download=True")
            dataset_path = download_dataset("carrie1/ecommerce-data", output_dir=data_dir, force_download=True)
        csv_path = find_first_csv(dataset_path)

    if csv_path:
        df = extract_data_csv(csv_path)
        logger.info("✓ EXTRACT AND READ CSV FILES IS SUCCESSFULLY")
        return df
    else:
        raise FileNotFoundError("❌ File does not exist or is not valid")
```

8.2 Apa Itu Orchestrator Function?

Orchestrator function (fungsi orkestrator) adalah fungsi yang **mengkoordinasikan** pemanggilan fungsi-fungsi lain. Fungsi ini tidak melakukan "pekerjaan berat" sendiri — ia hanya menentukan **urutan** dan **logika** pemanggilan.

Bayangkan analogi orkestra musik:

- 🎵 **Pemain biola, drum, piano** = fungsi-fungsi individual (`download_dataset` , `find_first_csv` , `extract_data_csv`)
- 🎼 **Konduktor** = fungsi orchestrator (`extract_get_data`) — menentukan siapa bermain kapan

```
extract_get_data() ← ORCHESTRATOR
├─ find_first_csv()    ← langkah 1: cari CSV lokal
├─ download_dataset()  ← langkah 2: download jika perlu
├─ find_first_csv()    ← langkah 3: cari CSV di hasil download
└─ extract_data_csv()  ← langkah 4: baca CSV menjadi DataFrame
```

8.3 Alur Logika Lengkap

Langkah 1: Tampilkan Header

```
print("="*60)
print("PHASE 1: EXTRACT CSV FORMATS")
print("="*60)
```

Menampilkan header visual di terminal agar user tahu fase mana yang sedang berjalan. `"="*60` menghasilkan 60 karakter `=` berturut-turut.

Output:

```
=====
PHASE 1: EXTRACT CSV FORMATS
=====
```

Langkah 2: Tentukan Path Data

```
data_dir = project_root / "data"
```

Menggabungkan `project_root` dengan `"data"` menggunakan operator `/` dari `pathlib`. Hasilnya misalnya: `C:\Users\korba\OneDrive\Documents\PROJECTS\ETL_Pipeline_Python\data`

Langkah 3: Cari CSV Lokal (Pengecekan Pertama)

```
csv_path = find_first_csv(data_dir)
```

Mencari file `.csv` pertama di folder `data/`. Jika ditemukan, `csv_path` berisi path ke file tersebut. Jika tidak ada file CSV, `find_first_csv()` akan melempar `ValueError`.

Langkah 4: Download Jika Perlu

```
if not csv_path:
    try:
        dataset_path = download_dataset("carrie1/ecommerce-data", output_dir=data_dir)
    except FileExistsError:
        logger.warning("⚠ Folder data is not empty. Retrying download with force_download=True")
        dataset_path = download_dataset("carrie1/ecommerce-data", output_dir=data_dir, force_download=True)
    csv_path = find_first_csv(dataset_path)
```

Jika `csv_path` kosong/None (tidak ada CSV lokal):

1. Coba download dari Kaggle
2. Jika folder `data/` sudah ada isinya (`FileExistsError`), coba lagi dengan `force_download=True`
3. Setelah download, cari file CSV di folder hasil download

8.4 Penanganan `FileExistsError` dan Retry

```
try:
    dataset_path = download_dataset("carrie1/ecommerce-data", output_dir=data_dir)
except FileExistsError:
    logger.warning("⚠ Folder data is not empty. Retrying download with force_download=True")
    dataset_path = download_dataset("carrie1/ecommerce-data", output_dir=data_dir, force_download=True)
```

Kapan `FileExistsError` terjadi?

`kagglehub.dataset_download()` bisa melempar `FileExistsError` ketika folder tujuan (`data/`) **sudah berisi file** (mungkin file non-CSV, file lama, atau file rusak).

Strategi Retry:

1. **Percobaan 1:** Download tanpa force (`force_download=False` — default)
2. **Jika gagal karena folder sudah ada isinya:** Log warning, lalu...
3. **Percobaan 2:** Download dengan force (`force_download=True`) — ini akan menimpa file yang ada

Ini adalah pola **retry with escalation** — kita mulai dengan pendekatan "sopan" (jangan timpa), dan baru "paksa" jika pendekatan pertama gagal.

8.5 Mengapa Ada Dua Pengecekan `csv_path` ?

```
# Pengecekan PERTAMA (baris 136)
csv_path = find_first_csv(data_dir) # Cek folder data/ lokal

if not csv_path:
    # ... download dari Kaggle ...
    # Pengecekan KEDUA (baris 145)
    csv_path = find_first_csv(dataset_path) # Cek folder hasil download

# Pengecekan KETIGA (baris 148)
if csv_path:
    df = extract_data_csv(csv_path)
    ...
else:
    raise FileNotFoundError(...)
```

Alasan:

| Pengecekan | Tujuan | Kapan Relevan | |-----|-----|-----| | **Pertama** (baris 136) | Cek apakah CSV sudah ada **secara lokal** | Saat user sudah punya data di folder `data/` | | **Kedua** (baris 145) | Cek apakah CSV ada **setelah download** | Saat data harus didownload dulu dari Kaggle | | **Ketiga** (baris 148) | Konfirmasi akhir bahwa kita **punya CSV** | Selalu — sebagai safety net sebelum membaca |

Ini menerapkan prinsip **defense in depth** — beberapa lapisan pengecekan untuk memastikan data benar-benar ada sebelum diproses.

9. Fungsi `summary_data()` — Eksplorasi Data Awal

9.1 Kode Lengkap

```
def summary_data(dataframe: pd.DataFrame) -> pd.DataFrame:
    data = dataframe
    if data is None:
        return
    print("\n")
    print("="*60)
    print("SUMMARY DATASET")
    print("="*60)
    print(f"Check Sample Data :\n{data.head()}\n")
    logger.info(f" Dataset Loaded - Rows: {dataframe.shape[0]}, Columns: {dataframe.shape[1]}")
    print(f"\n👁️ Check Data Types :\n{data.dtypes}")
    print("\n Total Missing Values")
    total_missing_values = dataframe.isnull().sum()
    for column, count in total_missing_values.items():
        if count > 0:
            print(f"Columns '{column}' has {count} missing values.")
```

9.2 `data = dataframe` dan Pengecekan None

```
data = dataframe
if data is None:
    return
```

- `data = dataframe` — membuat **alias** (nama lain) untuk `dataframe`. Variabel `data` dan `dataframe` menunjuk ke **objek yang sama** di memori. Ini dilakukan untuk kemudahan pengetikan (nama lebih pendek).
- `if data is None` — mengecek apakah DataFrame tidak ada. Jika `None`, fungsi langsung **return** tanpa melakukan apa-apa (return tanpa nilai = return `None`).

💡 **Catatan:** `is` berbeda dari `==` di Python. `is` mengecek **identitas objek** (apakah mereka objek yang sama), sedangkan `==` mengecek **kesamaan nilai**. Untuk mengecek `None`, selalu gunakan `is None` — ini adalah best practice Python.

9.3 `df.head()` — Menampilkan 5 Baris Pertama

```
print(f"Check Sample Data :\n{data.head()}\n")
```

Apa yang dilakukan `head()` ?

`head()` mengembalikan **5 baris pertama** dari DataFrame. Ini adalah cara cepat untuk **melihat sampel data** dan memastikan data terbaca dengan benar.

```
data.head()      # 5 baris pertama (default)
data.head(10)    # 10 baris pertama
data.head(1)     # 1 baris pertama
```

Contoh output:

Check Sample Data :

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	Cust
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	178
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	178
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	178
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	178
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	178

Mengapa penting?

Dengan melihat `head()`, kita bisa memverifikasi:

- ✓ Kolom terbaca dengan benar
- ✓ Data tidak korup
- ✓ Separator CSV benar (bukan semua data di satu kolom)
- ✓ Encoding benar (tidak ada karakter aneh)

9.4 `df.shape` — Dimensi Data (Baris dan Kolom)

```
logger.info(f"📊 Dataset Loaded - Rows: {dataframe.shape[0]}, Columns: {dataframe.shape[1]}")
```

Apa itu `shape` ?

`shape` adalah properti DataFrame yang mengembalikan **tuple** berisi `(jumlah_baris, jumlah_kolom)`.

```
dataframe.shape      # (541909, 8) → tuple
dataframe.shape[0]   # 541909 → jumlah baris
dataframe.shape[1]   # 8 → jumlah kolom
```

Mengapa penting?

- **Jumlah baris** menunjukkan berapa banyak data yang berhasil diekstrak. Jika terlalu sedikit, mungkin ada masalah saat pembacaan.
- **Jumlah kolom** menunjukkan berapa banyak fitur/atribut yang tersedia. Jika kurang dari yang diharapkan, mungkin ada kolom yang hilang.

9.5 `df.dtypes` — Tipe Data Setiap Kolom

```
print(f"\n👁️ Check Data Types :\n{data.dtypes}")
```

Apa itu `dtypes` ?

`dtypes` menampilkan **tipe data** setiap kolom dalam DataFrame.

Contoh output:

```
👁️ Check Data Types :
InvoiceNo      object
StockCode      object
Description    object
Quantity       int64
InvoiceDate    object
UnitPrice      float64
CustomerID     float64
Country        object
```

Tipe data pandas yang umum:

Tipe	Penjelasan	Contoh
<code>object</code>	String/teks (atau campuran tipe)	"536365", "WHITE HANGING..."
<code>int64</code>	Bilangan bulat 64-bit	6, 100, -3
<code>float64</code>	Bilangan desimal 64-bit	2.55, 17850.0
<code>bool</code>	Boolean	True, False
<code>datetime64</code>	Tanggal dan waktu	2010-12-01 08:26:00
<code>category</code>	Kategori (seperti enum) Hasil dari <code>astype('category')</code>	

Mengapa pengecekan tipe data penting?

- `InvoiceDate` bertipe `object` (bukan `datetime64`) — ini harus **dikonversi** di fase Transform!
- `CustomerID` bertipe `float64` — mungkin harus diubah ke `int64` atau `str` di fase Transform
- Mengetahui tipe data membantu kita merencanakan **transformasi** yang diperlukan

9.6 `df.isnull().sum()` — Menghitung Missing Values

```
total_missing_values = dataframe.isnull().sum()
for column, count in total_missing_values.items():
    if count > 0:
        print(f"Columns '{column}' has {count} missing values.")
```

Breakdown langkah per langkah:

```
# Langkah 1: dataframe.isnull()
# Menghasilkan DataFrame boolean – True jika nilai kosong/NaN, False jika ada nilai
#   InvoiceNo  StockCode  Description  Quantity  ...
# 0      False      False      False      False  ...
# 1      False      False      False      False  ...
# 2      False      False      True       False  ... ← Description kosong!

# Langkah 2: .sum()
# Menjumlahkan True (=1) per kolom → total missing values per kolom
# InvoiceNo      0
# StockCode      0
# Description    1454
# Quantity       0
# CustomerID    135080
# ...

# Langkah 3: Loop dan tampilkan yang > 0
for column, count in total_missing_values.items():
    if count > 0:
        print(f"Columns '{column}' has {count} missing values.")
```

Contoh output:

```
 Total Missing Values
Columns 'Description' has 1454 missing values.
Columns 'CustomerID' has 135080 missing values.
```

Mengapa pengecekan missing values penting sebelum Transform?

Missing values (NaN) bisa menyebabkan:

1. **Error saat kalkulasi** — misalnya menghitung rata-rata dengan NaN
2. **Hasil yang salah** — missing values bisa bias analisis

3. **Error saat Load** — beberapa database tidak menerima NULL tanpa handling khusus

Mengetahui missing values di fase Extract membantu kita **merencanakan strategi** penanganan di fase Transform (hapus baris, isi dengan default, interpolasi, dll).

10. Blok `if __name__ == "__main__"` — Penjelasan Mendalam

10.1 Kode Lengkap

```
if __name__ == "__main__":
    project_root = Path(__file__).resolve().parent.parent
    result_data = extract_get_data(project_root)
    summary_data(result_data)
```

10.2 Apa Artinya `__name__` dan `__main__` ?

`__name__` adalah variabel khusus (special variable) yang Python otomatis buat di setiap modul/file.

Nilai `__name__` tergantung **bagaimana file dijalankan**:

Cara Menjalankan	Nilai <code>__name__</code>	Contoh
Langsung (<code>python extract.py</code>)	<code>"__main__"</code>	<code>python src/extract.py</code>
Diimport (<code>import extract</code>)	<code>"extract"</code> (nama modul)	Di <code>main.py</code> : <code>from src import extract</code>

Jadi `if __name__ == "__main__":` artinya:

"Jalankan kode di bawah ini **HANYA JIKA** file ini dieksekusi secara langsung, BUKAN ketika diimport oleh file lain."

10.3 Mengapa Penting untuk Modularitas?

Tanpa guard ini, kode akan **selalu berjalan** — bahkan saat diimport:

```
# ❌ TANPA guard – Masalah!
# File: extract.py
project_root = Path(__file__).resolve().parent.parent
result_data = extract_get_data(project_root)
summary_data(result_data)

# File: main.py
from src import extract # ← Ini akan LANGSUNG menjalankan semua kode di atas!
# Padahal kita hanya mau import fungsinya saja...
```

```
# ✅ DENGAN guard – Aman!
# File: extract.py
if __name__ == "__main__":
    project_root = Path(__file__).resolve().parent.parent
    result_data = extract_get_data(project_root)
    summary_data(result_data)

# File: main.py
from src import extract # ← Kode di dalam if TIDAK dijalankan
# Kita bisa gunakan extract.extract_get_data() secara manual
```

Ini sangat penting dalam project ETL karena:

- File `extract.py` bisa dijalankan **standalone** untuk testing
- File `extract.py` juga bisa **diimport** oleh `main.py` atau file lain
- Tanpa guard, import akan memicu proses extract yang tidak diinginkan

10.4 `Path(__file__).resolve().parent.parent` — Navigasi Path

```
project_root = Path(__file__).resolve().parent.parent
```

Mari kita bedah langkah per langkah:

```
# Asumsikan file berada di: C:\Users\korba\...\ETL_Pipeline_Python\src\extract.py

__file__
# → "src/extract.py" (path relatif ke file saat ini)

Path(__file__)
# → Path("src/extract.py") (konversi string ke objek Path)

Path(__file__).resolve()
# → Path("C:/Users/korba/...\ETL_Pipeline_Python/src/extract.py")
# resolve() mengkonversi ke ABSOLUTE path (path lengkap dari root)

Path(__file__).resolve().parent
# → Path("C:/Users/korba/...\ETL_Pipeline_Python/src")
# parent = satu level ke atas = folder "src"

Path(__file__).resolve().parent.parent
# → Path("C:/Users/korba/...\ETL_Pipeline_Python")
# parent.parent = dua level ke atas = folder PROJECT ROOT
```

Visualisasi:

```
ETL_Pipeline_Python/      ← parent.parent (PROJECT ROOT) ✓
├── src/                  ← parent
│   ├── extract.py        ← __file__ (file saat ini)
│   ├── transform.py
│   └── load.py
├── data/
│   └── data.csv
├── documents/
└── tests/
```

Mengapa menggunakan `resolve()` ?

`resolve()` penting karena:

1. Mengkonversi path relatif menjadi **absolute** (path lengkap)
2. Menyelesaikan **symlink** (symbolic link) jika ada
3. Menghasilkan path yang **konsisten** di semua OS (Windows, Linux, macOS)

Tanpa `resolve()`, `__file__` bisa berisi path relatif seperti `"src/extract.py"`, dan `.parent.parent` mungkin tidak menghasilkan project root yang benar.

11. 🛡️ Error Handling Strategy

11.1 Daftar Semua Error yang Ditangani

Berikut adalah ringkasan **semua error** yang ditangani dalam source code:

Error	Di Fungsi	Baris	Penyebab	Aksi
<code>ValueError("Invalid File Format")</code>	<code>extract_data_csv</code>	79	File bukan <code>.csv</code>	Log error, raise
<code>FileNotFoundError("File Path Does Not Exist")</code>	<code>extract_data_csv</code>	84	File tidak ada di disk	Log error, raise
<code>UnicodeDecodeError</code>	<code>extract_data_csv</code>	92	Encoding UTF-8 gagal	Fallback ke ISO-8859-1
<code>pd.errors.EmptyDataError</code>	<code>extract_data_csv</code>	100	File CSV kosong	Log error, re-raise
<code>PermissionError</code>	<code>extract_data_csv</code>	105	OS menolak akses	Log error, re-raise
<code>Exception</code> (generic)	<code>extract_data_csv</code>	110	Error tak terduga	Log error, re-raise
<code>ValueError("No CSV files...")</code>	<code>find_first_csv</code>	54	Tidak ada file <code>.csv</code> di folder	Log error, raise
<code>FileExistsError</code>	<code>extract_get_data</code>	142	Folder tujuan download sudah ada	Retry dengan <code>force_download</code>
<code>FileNotFoundError("File does not...")</code>	<code>extract_get_data</code>	153	CSV tetap tidak ditemukan setelah semua upaya	Raise

11.2 Hierarki Exception di Python

Semua exception di Python mewarisi dari class `BaseException`. Berikut hierarki yang relevan dengan source code kita:

```

BaseException
├── Exception
│   ├── ValueError
│   │   ├── UnicodeDecodeError
│   │   └── OSError
│   │       ├── FileNotFoundError
│   │       ├── FileExistsError
│   │       └── PermissionError
│   └── pandas.errors.EmptyDataError

```

← caught by "except Exception as e"

← "Invalid File Format", "No CSV files..."

← Encoding gagal (warisan dari ValueError!)

← File tidak ditemukan

← File/folder sudah ada

← Tidak punya izin akses

← File CSV kosong (khusus pandas)

💡 **Fakta menarik:** `UnicodeDecodeError` adalah **subclass** dari `ValueError` ! Ini berarti `except ValueError` juga akan menangkap `UnicodeDecodeError`. Dalam source code kita, ini **tidak masalah** karena `UnicodeDecodeError` ditangkap di **try dalam** (lebih spesifik) sebelum `ValueError` di try luar.

11.3 Best Practices Error Handling dalam ETL

Berikut best practices yang **sudah diterapkan** dalam source code:

Best Practice	Penjelasan	Diterapkan?
Specific exceptions first	Tangkap exception spesifik sebelum generic	✅ <code>EmptyDataError</code> → <code>PermissionError</code> → <code>Exception</code>
Log before raise	Catat error ke log sebelum melempar kembali	✅ Semua error di-log sebelum raise
Don't swallow exceptions	Jangan abaikan error diam-diam	✅ Semua error di-raise kembali
Validate early	Validasi input sebelum proses	✅

berat | ☒ Cek `.suffix` dan `.exists()` sebelum `read_csv` || **Graceful degradation** | Coba alternatif sebelum gagal total | ☒ UTF-8 → ISO-8859-1, download → force download || **Meaningful error messages** | Pesan error harus deskriptif | ☒ Emoji + deskripsi jelas || **Fail fast** | Gagal segera jika ada masalah fundamental | ☒ Validasi file di awal fungsi |

Berikut best practices yang **bisa ditambahkan** di masa depan:

| Best Practice | Penjelasan | Status | |-----|-----|-----| | **Custom exceptions** | Buat exception class khusus untuk ETL | ☒ Belum || **Retry with backoff** | Retry dengan jeda waktu yang meningkat | ☒ Belum || **Timeout handling** | Batasi waktu download | ☒ Belum || **Circuit breaker** | Hentikan retry setelah N kali gagal | ☒ Belum || **Structured logging** | Log dalam format JSON untuk parsing otomatis | ☒ Belum |

12. Konsep Python Yang Digunakan

12.1 Type Hints

Type hints (petunjuk tipe) diperkenalkan di Python 3.5 melalui PEP 484. Ini memungkinkan kita mendeklarasikan tipe data yang diharapkan untuk parameter dan return value.

```
# Sintaks dasar:
# parameter: tipe
# -> tipe_return

def download_dataset(dataset: str, output_dir: Path, force_download: bool = False) -> pd.DataFrame:
#             ^^^^^^^^^      ^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^      ^^^^^^^^^^^^^^^^^
#             parameter1      parameter2              parameter3 + default      return type
```

Type hints yang digunakan di source code:

| Type Hint | Arti | Contoh Nilai | |-----|-----|-----| | `str` | String (teks) | `"carrie1/ecommerce-data"` || `Path` | Objek path dari pathlib | `Path("data/data.csv")` || `bool` | Boolean (benar/salah) | `True` , `False` || `pd.DataFrame` | DataFrame pandas | `pd.read_csv("data.csv")` |

Keuntungan type hints:

1. **IDE Support** — IDE seperti VS Code atau PyCharm bisa memberikan **autocompletion** dan **warning** jika tipe salah
2. **Dokumentasi Hidup** — Programmer lain langsung tahu tipe data yang diharapkan tanpa membaca docstring
3. **Static Analysis** — Tools seperti `mypy` bisa mengecek kesalahan tipe **sebelum** program dijalankan
4. **Self-documenting Code** — Kode menjadi lebih mudah dipahami

```
# ❌ Tanpa type hints – apa tipe dataset? apa tipe return?
def download_dataset(dataset, output_dir, force_download=False):
    ...

# ✅ Dengan type hints – jelas!
def download_dataset(dataset: str, output_dir: Path, force_download: bool = False) -> pd.DataFrame:
    ...
```

12.2 Default Parameters

Default parameters memungkinkan parameter memiliki nilai bawaan yang digunakan jika pemanggil tidak memberikan nilai.

[illegible]

Aturan penting:

1. Default parameters harus **di akhir** daftar parameter
2. Jangan gunakan objek **mutable** (list, dict) sebagai default

```
#  BENAR
def func(a, b, c=10):
    ...
    # c punya default, di akhir

#  SALAH
def func(a, b=10, c):
    ...
    # SyntaxError! default parameter di tengah

#  BERBAHAYA
def func(items=[]):
    items.append(1)
    return items
    # Mutable default! List ini dibagikan oleh semua pemanggil

#  AMAN
def func(items=None):
    if items is None:
        items = []
    items.append(1)
    return items
    # None sebagai sentinel value
```

12.3 Docstrings

Docstrings adalah string dokumentasi yang menjelaskan fungsi, class, atau modul. Diapit triple quotes (`"""`).

Format Google-style (yang digunakan di source code):

```
def extract_data_csv(file_path: Path) -> pd.DataFrame:
    """Extract data from CSV file.                                     ← Ringkasan singkat

    Args:                                                             ← Parameter
        file_path (Path): The path to the CSV file.

    Returns:                                                         ← Nilai return
        pd.DataFrame: DataFrame from the CSV file.
        None: If the file path is invalid or does not exist.

    Raises:                                                         ← Exception yang mungkin
        ValueError: If the file path is invalid or does not exist.
        FileNotFoundError: If the file path is invalid or does not exist.
    """
```

Format lain yang populer:


```
# NumPy-style
def func(param):
    """
    Ringkasan singkat.

    Parameters
    -----
    param : str
        Deskripsi parameter.

    Returns
    -----
    int
        Deskripsi return value.
    """

# Sphinx-style (reStructuredText)
def func(param):
    """
    Ringkasan singkat.

    :param param: Deskripsi parameter.
    :type param: str
    :returns: Deskripsi return value.
    :rtype: int
    """
```

12.4 f-strings vs `%s` Formatting

Dalam source code, ada **dua cara formatting string** yang digunakan:

f-strings (Formatted String Literals) — Python 3.6+

```
# Digunakan di baris seperti:
logger.info(f"📄 Download Dataset From Kaggle")
logger.info(f"📄 Reading CSV Files : {file_path.name}")
logger.info(f"📊 Dataset Loaded - Rows: {dataframe.shape[0]}, Columns: {dataframe.shape[1]}")
print(f"Check Sample Data : \n{data.head()}\n")
```

Cara kerja: Awali string dengan `f`, lalu masukkan ekspresi Python di dalam kurung kurawal `{}`.

```
name = "data.csv"
rows = 541909
print(f"File {name} memiliki {rows} baris")
# Output: File data.csv memiliki 541909 baris

# Bisa juga ekspresi:
print(f"Total: {100 * 3 + 5}") # Output: Total: 305
```

%s Formatting (Old-style) — Semua versi Python

```
# Digunakan di baris seperti:
logger.error("⚠️ No CSV files found in dataset path: %s", dataset_path)
logger.info("✓ Found CSV file: %s", csv_files[0])
```

Cara kerja: %s adalah placeholder yang diganti dengan argumen berikutnya. Di logging, argumen dipisahkan koma.

```
logger.info("File: %s, Baris: %d", "data.csv", 541909)
# Output: File: data.csv, Baris: 541909
```

Perbandingan:

Aspek f-string %s formatting (di logging)	----- ----- -----	Kecepatan Cepat
Sedikit lebih cepat di logging*	Readability ✓ Sangat mudah dibaca	😬 Agak kurang intuitif
Semua versi	Best practice logging 😬 Oke tapi boros	✓ Direkomendasikan

*Di logging, %s formatting **lebih efisien** karena string hanya di-format jika log benar-benar ditampilkan. Dengan f-string, string **selalu** di-format meskipun level logging terlalu rendah untuk ditampilkan.

```
# ✓ Efisien: string hanya diformat jika level >= INFO
logger.info("Found CSV: %s", csv_files[0])

# 😬 Kurang efisien: f-string SELALU diformatkan dulu
logger.info(f"Found CSV: {csv_files[0]}")
# String sudah diformatkan meskipun level diset ke WARNING (INFO tidak ditampilkan)
```

12.5 Path Operations

Berikut ringkasan semua operasi **Path** yang digunakan di source code:

```

from pathlib import Path

# 1. Membuat Path
path = Path("data/data.csv")
path = Path(__file__)

# 2. Menggabungkan path (operator /)
data_dir = project_root / "data"
# Setara dengan: Path(str(project_root) + "/data")

# 3. Mengambil parent directory
Path("a/b/c.txt").parent      # → Path("a/b")
Path("a/b/c.txt").parent.parent # → Path("a")

# 4. Mengambil nama file
Path("data/data.csv").name    # → "data.csv"

# 5. Mengambil ekstensi
Path("data/data.csv").suffix  # → ".csv"

# 6. Mengecek keberadaan
Path("data/data.csv").exists() # → True/False

# 7. Resolve ke absolute path
Path("data/data.csv").resolve() # → Path("C:/Users/.../data/data.csv")

# 8. Glob (mencari file dengan pola)
Path("data").glob("*.csv")     # → generator of matching paths

# 9. Konversi ke string
str(Path("data/data.csv"))     # → "data/data.csv" (atau "data\\data.csv" di Windows)

```

12.6 List Comprehension dan Iterasi

Meskipun tidak ada list comprehension eksplisit di source code, ada **iterasi** yang perlu dipahami:

```

# Iterasi dengan .items() – digunakan di summary_data()
for column, count in total_missing_values.items():
    if count > 0:
        print(f"Columns '{column}' has {count} missing values.")

```

Apa itu `.items()` ?

`.items()` mengembalikan pasangan (**key, value**) dari sebuah Series (atau Dictionary). Dalam konteks ini:

- `column` = nama kolom (key)
- `count` = jumlah missing values (value)

```
# Contoh manual:
total_missing_values = pd.Series({
    "InvoiceNo": 0,
    "Description": 1454,
    "CustomerID": 135080
})

for column, count in total_missing_values.items():
    print(f"{column}: {count}")

# Output:
# InvoiceNo: 0
# Description: 1454
# CustomerID: 135080
```

12.7 String Multiplication

```
print("="*60)
```

Dalam Python, operator `*` pada string menghasilkan **pengulangan**:

```
"=" * 5      # → "====="
"- " * 10     # → "- - - - -"
"abc" * 3     # → "abcabcabc"
"=" * 60     # → "=====
```

Ini digunakan untuk membuat **separator visual** di terminal output.

13. Ringkasan & Poin-Poin Penting

13.1 Checklist Apa yang Sudah Dipelajari

Setelah mempelajari dokumentasi ini secara menyeluruh, berikut checklist pengetahuan yang seharusnya sudah kamu kuasai:

Konsep ETL

- ☐ Memahami apa itu ETL (Extract, Transform, Load)
- ☐ Memahami mengapa Extract adalah fase pertama dan paling krusial
- ☐ Memahami berbagai sumber data yang bisa diekstrak (CSV, API, Database, dll.)
- ☐ Memahami prinsip-prinsip Extract yang baik (validasi, error handling, logging)

Konsep Python

- ☐ Memahami `import` dan penggunaan alias (`import pandas as pd`)
- ☐ Memahami **type hints** (`str` , `Path` , `bool` , `pd.DataFrame`)
- ☐ Memahami **default parameters** (`force_download: bool = False`)
- ☐ Memahami **docstrings** dan format Google-style
- ☐ Memahami **f-strings** dan `%s` formatting
- ☐ Memahami `if __name__ == "__main__"` dan modularitas
- ☐ Memahami **truthiness/falsiness** di Python (`if not csv_files`)
- ☐ Memahami **exception hierarchy** dan `raise` / `re-raise`

Library & Tools

- ☐ Memahami `pandas` untuk membaca CSV (`pd.read_csv()`)
- ☐ Memahami `logging` module (level, format string, logger instance)
- ☐ Memahami `kagglehub` untuk download dataset dari Kaggle
- ☐ Memahami `pathlib.Path` dan keunggulannya vs `os.path`

Error Handling

- ☐ Memahami `try-except` dan **nested try-except**
- ☐ Memahami perbedaan `raise` , `raise Exception(...)` , dan `raise ... from ...`
- ☐ Memahami **graceful degradation** (UTF-8 → ISO-8859-1)
- ☐ Memahami **retry pattern** (`FileExistsError` → `force_download=True`)
- ☐ Memahami mengapa **jangan menelan exception** (swallow exceptions)

Data Exploration

- ☐ Memahami `df.head()` — melihat sampel data
- ☐ Memahami `df.shape` — dimensi data
- ☐ Memahami `df.dtypes` — tipe data kolom
- ☐ Memahami `df.isnull().sum()` — menghitung missing values

- [] Memahami mengapa summary penting sebelum fase Transform

13.2 Tips dan Best Practices

💡 Tips Umum ETL

1. **Selalu validasi data di awal** — Jangan percaya data mentah. Cek format, encoding, keberadaan file, dan kelengkapan data.
2. **Gunakan logging, bukan print** — `print()` cocok untuk header visual, tapi untuk tracking proses gunakan `logging`. Di produksi, log bisa dikirim ke file atau sistem monitoring.
3. **Design for failure** — Asumsikan setiap operasi **bisa gagal**. Siapkan error handling yang komprehensif.
4. **Pisahkan concerns** — Setiap fungsi harus punya **satu tanggung jawab** (Single Responsibility Principle):
 - `download_dataset()` → hanya download
 - `find_first_csv()` → hanya cari file
 - `extract_data_csv()` → hanya baca CSV
 - `extract_get_data()` → hanya koordinasi
5. **Gunakan Path, bukan string** — `pathlib.Path` lebih aman, cross-platform, dan readable dibanding manipulasi string manual.

💡 Tips Error Handling

6. **Tangkap exception spesifik dulu** — Urutkan dari yang paling spesifik ke paling umum:

```
# ✅ BENAR
except pd.errors.EmptyDataError: ...
except PermissionError: ...
except Exception: ... # generic terakhir

# ❌ SALAH — Exception menangkap semua, yang di bawah tidak akan pernah tercapai
except Exception: ...
except PermissionError: ... # dead code!
```

7. **Log error sebelum re-raise** — Ini memastikan error tercatat meskipun pemanggil tidak menanganinya.
8. **Gunakan `raise` tanpa argumen untuk re-raise** — Ini mempertahankan traceback asli yang sangat berguna untuk debugging.

💡 Tips Encoding

9. **Default ke UTF-8, fallback ke Latin-1** — Ini adalah strategi yang solid karena UTF-8 adalah standar modern, dan ISO-8859-1 (Latin-1) adalah "universal decoder" yang tidak pernah gagal.
10. **Kenali data source kamu** — Jika kamu tahu data berasal dari sistem Eropa lama, mungkin langsung gunakan ISO-8859-1. Jika dari sistem modern, UTF-8 pasti benar.

💡 Tips untuk Langkah Selanjutnya

11. **Setelah Extract selesai, lihat summary data dengan seksama.** Missing values, tipe data yang salah, dan anomali lainnya harus dicatat untuk ditangani di fase **Transform**.
12. **Jaga raw data tetap utuh.** Jangan pernah mengubah file CSV asli. Semua transformasi harus dilakukan pada DataFrame di memori, dan hasilnya disimpan sebagai file baru atau di database.
13. **Pertimbangkan data versioning.** Untuk project yang lebih besar, gunakan tools seperti DVC (Data Version Control) untuk melacak versi dataset.

📝 Catatan Akhir:

Dokumentasi ini adalah bagian dari **learning journey** ETL Pipeline. Setelah memahami fase Extract, lanjutkan ke fase **Transform** dan **Load** untuk memahami pipeline secara keseluruhan.

Ingat: **Extract** mengambil data mentah → **Transform** membersihkan dan mengubah data → **Load** menyimpan data ke tujuan akhir.

Dokumentasi ini dibuat sebagai materi pembelajaran. Selamat belajar! 🚀